Errors Are Not Choke Points: poori research ek kahani mein

Harsh Dubey, Harshit Saini, Prashant Jain, Himanshu Kumar, Gaurav Kaushal, Rohit Chaudhary (supervisor) · KIET, CSE (AI&ML)

Simple Hinglish guide. Ise padhne ke baad paper (`paper/main.pdf`) ek baar padho, sab samajh aa jayega.

1 Shuruaat: problem kya tha?

LLM (jaise Qwen, Llama) maths ka sawaal step-by-step solve karta hai. Isko **chain of thought (CoT)** ya **trace** kehte hain. Kabhi beech mein ek step galat ho jaata hai aur final answer galat aa jaata hai. Poori field ka common idea yeh hai:

1. Pehla galat step dhundho (isko **first error** kehte hain; human annotators isko mark karte hain).
2. Wahan tak wapas jao aur wahan se dobara generate karo (**backtracking / repair**).

Process Reward Models (PRMs) isi kaam ke liye train hote hain, aur ProcessBench jaise benchmarks isi ko grade karte hain: "kya tumne pehla galat step pakda?" Hamara original aim tha **early failure detection**: kya hum trace ke shuru mein hi bata sakte hain ki yeh fail hoga, taaki jaldi intervene kar sakein?

Phase	Kahani ek nazar mein
1. Early detection	detector bana, par woh sirf sawaal ki difficulty padh raha tha (negative result)
2. Naya sawaal, v_k	failure kahan decide hota hai, model ke apne continuations se mapa
3. Main result	error $\neq$ choke point; repair, restart ko same compute par nahi harata; theory se proof
4. ChokeBench	naya benchmark; check kiya ki yeh asli recovery predict karta hai
5. RAG (secondary)	hallucination par same idea; result grader par depend karta hai, human rating baaki
6. Review aur refocus	do reviews ke baad claims narrow kiye, kamzor hisse imaandari se likhe
7. Run D	reasoning models (Qwen3, R1) par exact on-policy test, Kaggle par chal raha hai

2 Phase 1: early detection, aur pehla jhatka

ProcessBench (3,400 solutions, 12 models, har solution mein human-marked first error) par humne detectors banaye. Number achhe lag rahe the, lekin phir humne ek simple **baseline** lagaya: sirf sawaal ka text aur kaunsa model hai, trace ka ek bhi step dekhe bina.

Detector ko kya diya	AUROC	Matlab
Sirf sawaal + model (koi step nahi)	0.588	sawaal kitna mushkil hai, bas itna pata
+ trace ke pehle 25% steps	+0.005 (n.s.)	early steps se lagbhag kuch nahi milta
+ poora trace	+0.031	thoda signal, par tab tak bahut der ho chuki

Seekh (negative result)

Early detector asal mein *"sawaal mushkil hai"* padh raha tha, *"yeh attempt fail hoga"* nahi. Isko **difficulty confound** kehte hain. Yahi hamara pehla pivot tha: naya sawaal poocho.

3 Phase 2: naya sawaal, aur ek hi measurement v_k

Naya sawaal: **failure asal mein kahan "decide" hota hai, aur kya wahan se repair karna restart se behtar hai?** Iske liye ek quantity:

$$v_k = P(\text{sahi answer} \mid \text{sawaal} + \text{pehle } k \text{ steps})$$

Isko estimate karne ka tareeka: trace ko step k par kaato, *usi model* se wahan se 8–16 baar aage likhwao (**continuations / rollouts**), aur gino kitni baar answer sahi aaya. Total 125,832 continuations chalaye (Kaggle ke free T4 GPUs par).

Term	Simple matlab
v_0	bilkul shuru se solve karne ka chance. Yahan se dobara likhna = **restart**.
Backtrack to step j	step j se just pehle tak rakho, wahan se dobara likho (yaani v_{j-1} wala point).
Choke point (decisive step)	woh step jahan v sabse zyada girta hai. Yahan failure asal mein decide hota hai.
First error	woh step jahan human kehta hai "yahan galti hui".
Capability-limited	$v_0 \leq 1/8$: model ke liye sawaal shuru se hi lagbhag namumkin.

Example: $v = [0.6, 0.6, 0.1, 0.0]$. Step 2 par v 0.6 se 0.1 gira, toh choke point step 2 hai. Agar human ne step 3 ko galat mark kiya, toh *error* aur *choke point* alag hain.

4 Phase 3: kya mila (main results, maths)

Sawaal	Jawab (number)	Matlab
Kitne failures step 1 se pehle hi decide?	33–66% (4 models)	repair karne ko koi step hi nahi
Baaki mein human error = choke point?	0.438–0.647 (chance 0.16–0.20)	chance se behtar, par aadha hi sahi
Naye model (Llama-3.1-8B) par confirm?	haan, 0.633	preregistered test, pass
PRM kya pakadta hai? (ChokeBench)	human error 73%, choke point sirf 45%	verifier label pakadta hai, asli point nahi
Choke point ke liye train kiya model?	39%	better nahi hua
Restart kitna bachata hai? (4 attempts)	44–66% failures solve	simple restart bahut strong
PRM backtracking vs restart (same compute, Llama)	-0.063 (confirmed)	repair, restart se *haara*
Free perfect oracle vs restart (4× budget)	sirf +0.028	jaadu ki chhadi bhi bas thoda jeetti

Matched compute ka matlab: har method ko barabar GPU budget (tokens, PRM ki cost sab gini gayi). Tabhi comparison fair hai.

Kyun? (theory, Proposition 1: supermartingale)

Ek failed trace ka koi bhi point, jo aage dekhe bina chuna gaya ho, average mein shuru se behtar nahi hota: $E[v_\tau \mid \text{fail}] \leq v_0$. Simple bhasha: *jo trace fail hua, uska aadha hissa ek fresh start se zyada keemti nahi hota*. Isliye choke point dhundhne se tokens bachte hain, success nahi badhti. (Proposition 2: iska fayda bas chhote attempts hai, budget ka lagbhag 1.25–1.4×.)

5 ChokeBench: hamara naya benchmark

ProcessBench poochta hai "pehla galat step pakda?". **ChokeBench v1** (230 failed traces, 4 models) *dono* poochta hai: human error pakda, *aur* choke point pakda? Aur woh step resampling ke liye kitna keemti hai? Sabse important check: **kya ChokeBench score asli recovery predict karta hai?** (circular na ho, isliye samples ke aadhe hisse se choke point label kiya, baaki aadhe se gain mapa.)

Check (exploratory)	Result
Named step = choke point: recovery gain mein badhat	+0.40
Named step = human error, choke point ko control karne ke baad	+0.01 (n.s.)
Naye (held-out) model par ranking, Spearman ρ: ChokeBench vs ProcessBench-style	+0.52 vs +0.22

Matlab: human label tabhi kaam aata hai jab woh choke point se match kare. ChokeBench asli fayde ko behtar predict karta hai.

6 Phase 4 (secondary): hallucination wala RAG study

RAG mein model ko ek source diya jaata hai aur answer source se hi aana chahiye. Source mein jo nahi hai woh likh diya toh **hallucination**. Data: RAGTruth (human labels), model Llama-2-7B. Detector D = LettuceDetect (har sentence ko score karta hai).

Method	Cost (base = 1×)	Judged hallucination change vs base (E1 judge)
DRA: detect, restart, abstain	1.36×	-0.042 (exploratory)
CARE (choke-point-aware controller)	1.77×	-0.047 ($p = 0.0003$, preregistered)
CARE vs plain restart	–	-0.005 (koi fark nahi)
Chain-of-Verification	3.75×	mehenga, aur behtar nahi

Proposition 3 (law): restart loop jo answer deta hai, uski hallucination rate = detector ka $P(\text{hallucinated} \mid \text{pass})$, retries kitne bhi ho. Retries sirf coverage badhaate hain. Human labels se prediction: 50.2% → 26.6%. Check: 4× retries par coverage +2.4 points, hallucination lagbhag same (+0.6 se +1.5). Prediction sahi nikli.

Imaandari: yeh part kamzor kyun hai

(1) Judge E1 humans se kam agree karta hai ($\kappa = 0.219$; 81% drafts flag karta hai jab humans 50%). (2) Grader badlo toh result badal jaata hai: detector jaisa grader +0.326, independent judge +0.017 (n.s.). Gate khud ko grade kar raha tha. (3) Outputs ko abhi tak humans ne rate nahi kiya. Isliye RAG *secondary* hai, aur hum "hallucination solve" ka claim **nahi** karte.

7 Imaandari ke kadam (reviewers yahi dekhte hain)

Kya kiya	Kyun
Preregistration	hypotheses data se pehle git mein commit kiye, taaki baad mein result chun na sakein
PRE / EXP / CONF tags	har claim ke saath likha: pehle se plan tha, baad mein dekha, ya naye data par confirm hua
Sample splitting	ek half se chuno, doosre se score karo; warna restart nakli tareeke se achha dikhta
Bootstrap CIs, Holm correction	har number ke saath uncertainty; kai tests ki galti control
Mistakes withdraw kiye	"dual judge" aur "1,500-item" table galat the, hata diye, corrections log mein likha
Blinded human kits	base vs DRA ke 80 pairs, 2 raters; key ka hash pehle commit, key repo se bahar
External review	do reviews ke baad focus narrow kiya: main claim maths wala, RAG secondary

8 Run D: naya bada experiment (abhi Kaggle par chal raha hai)

Kyun? Pichhle traces ProcessBench ke the, kisi aur ne likhe the, isliye sirf "near on-policy" the. Proposition 1 exactly on-policy traces par hold karti hai. Aur aaj ki field ka sawaal *reasoning models* ka hai (jo hazaaron tokens sochte hain, `<think>` ke andar khud backtrack karte hain). Run D yahi test karta hai.

Setting	Value
Models	Qwen3-1.7B (thinking mode), DeepSeek-R1-Distill-Qwen-1.5B
Data	MATH-500, problems 21–500 (480; pehle 20 sirf pilot ke liye, analysis se bahar)
Stage 1	har problem ka ek trace, max 6,144 tokens; math-verify se grade
Stage 2	har failed trace ko 9 jagah kaato: 0, 256, 512, 1024, 1536, 2048, 3072, 4096, 5120 tokens (paragraph break par snap, taaki cut sirf past par depend kare); har cut se $m = 8$ continuations
Hardware	Kaggle, 2× T4 GPU, vLLM; har model ka ∼6.5 ghante ka session

Hypothesis (preregistered)	Test
D1: Proposition 1 exact on-policy par	har cut t par $v_t - v_0 \leq 0$? Agar kisi cut par significantly positive (Holm $p < 0.05$), toh theory refute
D2: koi cut restart ko 5 points se nahi harata	same token budget par best cut (half A se chuna, half B par score) minus restart; 4× par upper CI $< +0.05$
D3: descriptive	kitne failures shuru se decided ($v_0 \leq 1/8$), aur v sabse zyada kahan girta hai
D4: sensitivity	D1, D2 bina un traces ke jo token limit par kat gaye

Ab tak: pilot (8 problems) mein pipeline sahi chali; 6/8 sahi, dono failures token limit par kate. Speed kam thi (∼130 tokens/s per GPU), isliye fast CUDA-graph mode lagaya, $m = 8$ rakha, aur problems dataset order mein time tak process honge. Yeh sab **deviations data aane se pehle** preregistration doc mein likhe. Qwen3 ka run chal raha hai (raat 2:40 baje khatam), uske baad R1 shuru hoga (Kaggle ek waqt mein ek hi slot de raha hai). Result jo bhi aaye (theory sahi ya galat), paper mein waise hi jayega. Agar D2 mein koi cut restart ko harata hai, toh hum likhenge ki reasoning models conclusion badalte hain.

9 Ant mein: ek line mein research

Final takeaway

Errors are not choke points. Jahan human galti dikhata hai, failure aksar wahan decide nahi hota; bahut failures shuru se hi decided hote hain; verifiers label pakadte hain, asli point nahi; aur us point se repair karna, same compute par, simple restart ko nahi harata. Isliye restart woh baseline hai jise har localise-and-repair method ko harana chahiye. Hamara naya ChokeBench yeh test karne ka tool hai.

Abhi baaki: (1) Run D ke results; (2) human ratings (base vs DRA, 2 log, 2–3 ghante each); (3) doosre researchers ka replication. **Paper padhne ka order:** Abstract → Intro ke 5 contributions → Sec. 5–7 (counterfactual, policies, ChokeBench) → Sec. 8–9 (RAG) → Limitations → Appendix (proofs).